

Doloso_ACT1

October 7, 2026

0.1 Regression Model

```
[1]: import tensorflow as tf
print(tf.__version__) # check the version (should be 2.x+)

import datetime
print(f"Notebook last run (end-to-end): {datetime.datetime.now()}")
```

2.21.0

Notebook last run (end-to-end): 2026-10-07 21:49:24.666566

0.2 Making a new Data Set

```
[3]: import numpy as np
import matplotlib.pyplot as plt

# Make a bigger dataset
X = np.arange(-200, 200, 4)
X
```

```
[3]: array([-200, -196, -192, -188, -184, -180, -176, -172, -168, -164, -160,
          -156, -152, -148, -144, -140, -136, -132, -128, -124, -120, -116,
          -112, -108, -104, -100, -96, -92, -88, -84, -80, -76, -72,
          -68, -64, -60, -56, -52, -48, -44, -40, -36, -32, -28,
          -24, -20, -16, -12, -8, -4, 0, 4, 8, 12, 16,
           20, 24, 28, 32, 36, 40, 44, 48, 52, 56, 60,
           64, 68, 72, 76, 80, 84, 88, 92, 96, 100, 104,
          108, 112, 116, 120, 124, 128, 132, 136, 140, 144, 148,
          152, 156, 160, 164, 168, 172, 176, 180, 184, 188, 192,
          196])
```

```
[4]: # Make labels for the dataset (adhering to the same pattern as before)
y = np.arange(-180, 220, 4)
y
```

```
[4]: array([-180, -176, -172, -168, -164, -160, -156, -152, -148, -144, -140,
          -136, -132, -128, -124, -120, -116, -112, -108, -104, -100, -96,
           -92, -88, -84, -80, -76, -72, -68, -64, -60, -56, -52,
```

```

-48, -44, -40, -36, -32, -28, -24, -20, -16, -12, -8,
-4, 0, 4, 8, 12, 16, 20, 24, 28, 32, 36,
40, 44, 48, 52, 56, 60, 64, 68, 72, 76, 80,
84, 88, 92, 96, 100, 104, 108, 112, 116, 120, 124,
128, 132, 136, 140, 144, 148, 152, 156, 160, 164, 168,
172, 176, 180, 184, 188, 192, 196, 200, 204, 208, 212,
216])

```

```

[5]: # Same result as above
y = X + 20
y

```

```

[5]: array([-180, -176, -172, -168, -164, -160, -156, -152, -148, -144, -140,
-136, -132, -128, -124, -120, -116, -112, -108, -104, -100, -96,
-92, -88, -84, -80, -76, -72, -68, -64, -60, -56, -52,
-48, -44, -40, -36, -32, -28, -24, -20, -16, -12, -8,
-4, 0, 4, 8, 12, 16, 20, 24, 28, 32, 36,
40, 44, 48, 52, 56, 60, 64, 68, 72, 76, 80,
84, 88, 92, 96, 100, 104, 108, 112, 116, 120, 124,
128, 132, 136, 140, 144, 148, 152, 156, 160, 164, 168,
172, 176, 180, 184, 188, 192, 196, 200, 204, 208, 212,
216])

```

0.3 Splitting data into training and test sets

```

[6]: len(X)

```

```

[6]: 100

```

```

[8]: # Split data into train and test sets
X_train = X[:80] # first 40 examples (80% of data)
y_train = y[:80]

X_test = X[80:] # last 10 examples (20% of data)
y_test = y[80:]

len(X_train), len(X_test)

X_train, X_test

```

```

[8]: (array([-200, -196, -192, -188, -184, -180, -176, -172, -168, -164, -160,
-156, -152, -148, -144, -140, -136, -132, -128, -124, -120, -116,
-112, -108, -104, -100, -96, -92, -88, -84, -80, -76, -72,
-68, -64, -60, -56, -52, -48, -44, -40, -36, -32, -28,
-24, -20, -16, -12, -8, -4, 0, 4, 8, 12, 16,
20, 24, 28, 32, 36, 40, 44, 48, 52, 56, 60,
64, 68, 72, 76, 80, 84, 88, 92, 96, 100, 104,

```

```

    108, 112, 116]),
array([120, 124, 128, 132, 136, 140, 144, 148, 152, 156, 160, 164, 168,
       172, 176, 180, 184, 188, 192, 196]))

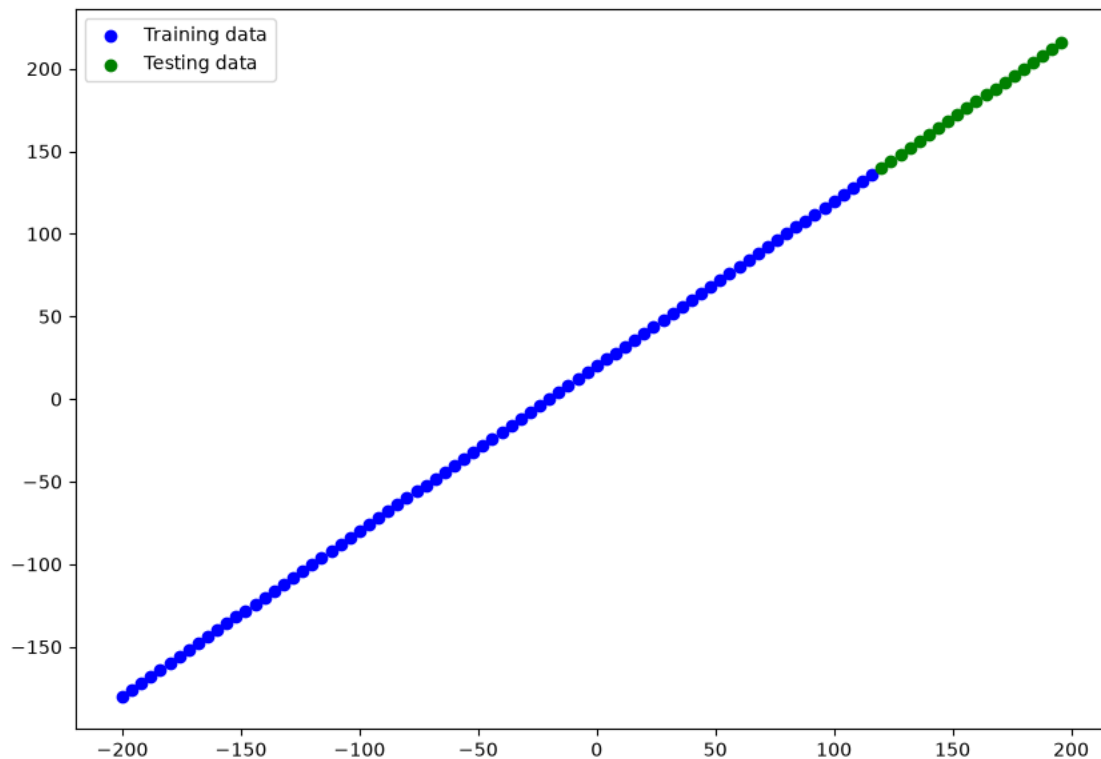
```

0.4 Visualizing the Data

```

[9]: plt.figure(figsize=(10, 7))
     # Plot training data in blue
     plt.scatter(X_train, y_train, c='b', label='Training data')
     # Plot test data in green
     plt.scatter(X_test, y_test, c='g', label='Testing data')
     # Show the legend
     plt.legend();

```



```

[10]: # Set random seed
      tf.random.set_seed(42)

      # Create a model (same as above)
      model = tf.keras.Sequential([
          tf.keras.layers.Dense(1)
      ])

```

```

# Compile model (same as above)
model.compile(loss=tf.keras.losses.mae,
              optimizer=tf.keras.optimizers.SGD(),
              metrics=["mae"])

# Fit model (same as above)
#model.fit(X_train, y_train, epochs=100) # commented out on purpose (not
↳ fitting it just yet)

```

WARNING:tensorflow:TensorFlow GPU support is not available on native Windows for TensorFlow >= 2.11. Even if CUDA/cuDNN are installed, GPU will not be used. Please use WSL2 or the TensorFlow-DirectML plugin.

0.5 Summary of Model

```
[11]: model.summary()
```

Model: "sequential"

Layer (type)	Output Shape	Param #
dense (Dense)	?	0 (unbuilt)

Total params: 0 (0.00 B)

Trainable params: 0 (0.00 B)

Non-trainable params: 0 (0.00 B)

0.6 Testing with Input Shape

```

[12]: # Set random seed
tf.random.set_seed(42)

# Create a model (same as above)
model = tf.keras.Sequential([
    tf.keras.layers.Input(shape=[1]),
    tf.keras.layers.Dense(5, activation='relu'),
    tf.keras.layers.Dense(1) # define the input_shape to our model
])

# Compile model (same as above)
model.compile(loss=tf.keras.losses.mae,

```

```
optimizer=tf.keras.optimizers.SGD(),  
metrics=["mae"])
```

```
[13]: model.summary()
```

Model: "sequential_1"

Layer (type)	Output Shape	Param #
dense_1 (Dense)	(None, 5)	10
dense_2 (Dense)	(None, 1)	6

Total params: 16 (64.00 B)

Trainable params: 16 (64.00 B)

Non-trainable params: 0 (0.00 B)

0.7 Fitting the model with training data

```
[14]: # Fit the model to the training data  
model.fit(X_train, y_train, epochs=100, verbose=1) # verbose controls how much  
↳ gets output
```

```
Epoch 1/100  
3/3          1s 43ms/step - loss:  
91.6989 - mae: 91.6989  
Epoch 2/100  
3/3          0s 34ms/step - loss:  
43.4403 - mae: 43.4403  
Epoch 3/100  
3/3          0s 31ms/step - loss:  
35.2027 - mae: 35.2027  
Epoch 4/100  
3/3          0s 37ms/step - loss:  
35.2815 - mae: 35.2815  
Epoch 5/100  
3/3          0s 32ms/step - loss:  
32.6413 - mae: 32.6413  
Epoch 6/100  
3/3          0s 43ms/step - loss:  
32.1632 - mae: 32.1632
```

Epoch 7/100
3/3 0s 31ms/step - loss:
40.5002 - mae: 40.5002
Epoch 8/100
3/3 0s 42ms/step - loss:
38.8891 - mae: 38.8891
Epoch 9/100
3/3 0s 39ms/step - loss:
33.5669 - mae: 33.5669
Epoch 10/100
3/3 0s 31ms/step - loss:
33.3083 - mae: 33.3083
Epoch 11/100
3/3 0s 30ms/step - loss:
32.8705 - mae: 32.8705
Epoch 12/100
3/3 0s 31ms/step - loss:
43.9568 - mae: 43.9568
Epoch 13/100
3/3 0s 51ms/step - loss:
39.1481 - mae: 39.1481
Epoch 14/100
3/3 0s 40ms/step - loss:
37.7722 - mae: 37.7722
Epoch 15/100
3/3 0s 29ms/step - loss:
32.6849 - mae: 32.6849
Epoch 16/100
3/3 0s 35ms/step - loss:
31.3375 - mae: 31.3375
Epoch 17/100
3/3 0s 29ms/step - loss:
41.3390 - mae: 41.3390
Epoch 18/100
3/3 0s 36ms/step - loss:
38.9144 - mae: 38.9144
Epoch 19/100
3/3 0s 29ms/step - loss:
34.1790 - mae: 34.1790
Epoch 20/100
3/3 0s 28ms/step - loss:
32.5335 - mae: 32.5335
Epoch 21/100
3/3 0s 31ms/step - loss:
30.4834 - mae: 30.4834
Epoch 22/100
3/3 0s 22ms/step - loss:
40.0960 - mae: 40.0960

Epoch 23/100
3/3 0s 29ms/step - loss:
38.1759 - mae: 38.1759
Epoch 24/100
3/3 0s 37ms/step - loss:
35.7506 - mae: 35.7506
Epoch 25/100
3/3 0s 52ms/step - loss:
32.0273 - mae: 32.0273
Epoch 26/100
3/3 0s 33ms/step - loss:
29.7648 - mae: 29.7648
Epoch 27/100
3/3 0s 35ms/step - loss:
36.9395 - mae: 36.9395
Epoch 28/100
3/3 0s 56ms/step - loss:
35.3270 - mae: 35.3270
Epoch 29/100
3/3 0s 36ms/step - loss:
31.5015 - mae: 31.5015
Epoch 30/100
3/3 0s 39ms/step - loss:
29.6282 - mae: 29.6282
Epoch 31/100
3/3 0s 29ms/step - loss:
42.6282 - mae: 42.6282
Epoch 32/100
3/3 0s 27ms/step - loss:
38.5228 - mae: 38.5228
Epoch 33/100
3/3 0s 26ms/step - loss:
34.6058 - mae: 34.6058
Epoch 34/100
3/3 0s 28ms/step - loss:
33.8675 - mae: 33.8675
Epoch 35/100
3/3 0s 30ms/step - loss:
28.6656 - mae: 28.6656
Epoch 36/100
3/3 0s 35ms/step - loss:
41.0902 - mae: 41.0902
Epoch 37/100
3/3 0s 27ms/step - loss:
36.0987 - mae: 36.0987
Epoch 38/100
3/3 0s 27ms/step - loss:
31.0224 - mae: 31.0224

Epoch 39/100
3/3 0s 28ms/step - loss:
30.6919 - mae: 30.6919
Epoch 40/100
3/3 0s 26ms/step - loss:
33.8046 - mae: 33.8046
Epoch 41/100
3/3 0s 29ms/step - loss:
41.4405 - mae: 41.4405
Epoch 42/100
3/3 0s 26ms/step - loss:
41.1069 - mae: 41.1069
Epoch 43/100
3/3 0s 27ms/step - loss:
35.3489 - mae: 35.3489
Epoch 44/100
3/3 0s 50ms/step - loss:
33.7103 - mae: 33.7103
Epoch 45/100
3/3 0s 40ms/step - loss:
28.9476 - mae: 28.9476
Epoch 46/100
3/3 0s 34ms/step - loss:
41.3330 - mae: 41.3330
Epoch 47/100
3/3 0s 36ms/step - loss:
36.8366 - mae: 36.8366
Epoch 48/100
3/3 0s 28ms/step - loss:
34.1506 - mae: 34.1506
Epoch 49/100
3/3 0s 28ms/step - loss:
31.4388 - mae: 31.4388
Epoch 50/100
3/3 0s 26ms/step - loss:
31.5714 - mae: 31.5714
Epoch 51/100
3/3 0s 29ms/step - loss:
42.3684 - mae: 42.3684
Epoch 52/100
3/3 0s 29ms/step - loss:
40.2476 - mae: 40.2476
Epoch 53/100
3/3 0s 43ms/step - loss:
34.6517 - mae: 34.6517
Epoch 54/100
3/3 0s 30ms/step - loss:
33.1380 - mae: 33.1380

Epoch 55/100
3/3 0s 29ms/step - loss:
27.3843 - mae: 27.3843
Epoch 56/100
3/3 0s 26ms/step - loss:
41.5345 - mae: 41.5345
Epoch 57/100
3/3 0s 27ms/step - loss:
39.1163 - mae: 39.1163
Epoch 58/100
3/3 0s 28ms/step - loss:
35.3843 - mae: 35.3843
Epoch 59/100
3/3 0s 38ms/step - loss:
32.6795 - mae: 32.6795
Epoch 60/100
3/3 0s 26ms/step - loss:
30.7857 - mae: 30.7857
Epoch 61/100
3/3 0s 29ms/step - loss:
34.8594 - mae: 34.8594
Epoch 62/100
3/3 0s 46ms/step - loss:
34.2965 - mae: 34.2965
Epoch 63/100
3/3 0s 29ms/step - loss:
27.6607 - mae: 27.6607
Epoch 64/100
3/3 0s 50ms/step - loss:
31.2479 - mae: 31.2479
Epoch 65/100
3/3 0s 31ms/step - loss:
35.4804 - mae: 35.4804
Epoch 66/100
3/3 0s 37ms/step - loss:
30.4968 - mae: 30.4968
Epoch 67/100
3/3 0s 28ms/step - loss:
30.2693 - mae: 30.2693
Epoch 68/100
3/3 0s 33ms/step - loss:
34.8087 - mae: 34.8087
Epoch 69/100
3/3 0s 27ms/step - loss:
39.3976 - mae: 39.3976
Epoch 70/100
3/3 0s 34ms/step - loss:
40.5300 - mae: 40.5300

Epoch 71/100
3/3 0s 56ms/step - loss:
34.8845 - mae: 34.8845
Epoch 72/100
3/3 0s 27ms/step - loss:
33.3318 - mae: 33.3318
Epoch 73/100
3/3 0s 27ms/step - loss:
27.6353 - mae: 27.6353
Epoch 74/100
3/3 0s 30ms/step - loss:
27.9135 - mae: 27.9135
Epoch 75/100
3/3 0s 34ms/step - loss:
35.5149 - mae: 35.5149
Epoch 76/100
3/3 0s 30ms/step - loss:
33.8892 - mae: 33.8892
Epoch 77/100
3/3 0s 25ms/step - loss:
30.2633 - mae: 30.2633
Epoch 78/100
3/3 0s 31ms/step - loss:
35.7057 - mae: 35.7057
Epoch 79/100
3/3 0s 27ms/step - loss:
39.4431 - mae: 39.4431
Epoch 80/100
3/3 0s 29ms/step - loss:
39.5812 - mae: 39.5812
Epoch 81/100
3/3 0s 26ms/step - loss:
34.8914 - mae: 34.8914
Epoch 82/100
3/3 0s 26ms/step - loss:
30.0086 - mae: 30.0086
Epoch 83/100
3/3 0s 26ms/step - loss:
28.2223 - mae: 28.2223
Epoch 84/100
3/3 0s 30ms/step - loss:
40.2827 - mae: 40.2827
Epoch 85/100
3/3 0s 26ms/step - loss:
38.2961 - mae: 38.2961
Epoch 86/100
3/3 0s 29ms/step - loss:
36.7749 - mae: 36.7749

```
Epoch 87/100
3/3          0s 26ms/step - loss:
32.1379 - mae: 32.1379
Epoch 88/100
3/3          0s 28ms/step - loss:
28.4003 - mae: 28.4003
Epoch 89/100
3/3          0s 31ms/step - loss:
41.6886 - mae: 41.6886
Epoch 90/100
3/3          0s 27ms/step - loss:
39.2513 - mae: 39.2513
Epoch 91/100
3/3          0s 38ms/step - loss:
35.5004 - mae: 35.5004
Epoch 92/100
3/3          0s 36ms/step - loss:
32.7550 - mae: 32.7550
Epoch 93/100
3/3          0s 38ms/step - loss:
30.2681 - mae: 30.2681
Epoch 94/100
3/3          0s 34ms/step - loss:
33.6193 - mae: 33.6193
Epoch 95/100
3/3          0s 32ms/step - loss:
39.4453 - mae: 39.4453
Epoch 96/100
3/3          0s 48ms/step - loss:
39.6442 - mae: 39.6442
Epoch 97/100
3/3          0s 33ms/step - loss:
34.9017 - mae: 34.9017
Epoch 98/100
3/3          0s 26ms/step - loss:
34.5649 - mae: 34.5649
Epoch 99/100
3/3          0s 27ms/step - loss:
28.1966 - mae: 28.1966
Epoch 100/100
3/3          0s 34ms/step - loss:
41.2228 - mae: 41.2228
```

[14]: <keras.src.callbacks.history.History at 0x2047e5f2030>

0.8 Check Summary

```
[15]: model.summary()
```

Model: "sequential_1"

Layer (type)	Output Shape	Param #
dense_1 (Dense)	(None, 5)	10
dense_2 (Dense)	(None, 1)	6

Total params: 18 (76.00 B)

Trainable params: 16 (64.00 B)

Non-trainable params: 0 (0.00 B)

Optimizer params: 2 (12.00 B)

```
[18]: from tensorflow.keras.utils import plot_model  
plot_model(model, show_shapes=True)
```

You must install pydot (`pip install pydot`) for `plot\_model` to work.

0.9 Visualize the predictions

```
[19]: y_preds = model.predict(X_test)  
y_preds
```

1/1 1s 574ms/step

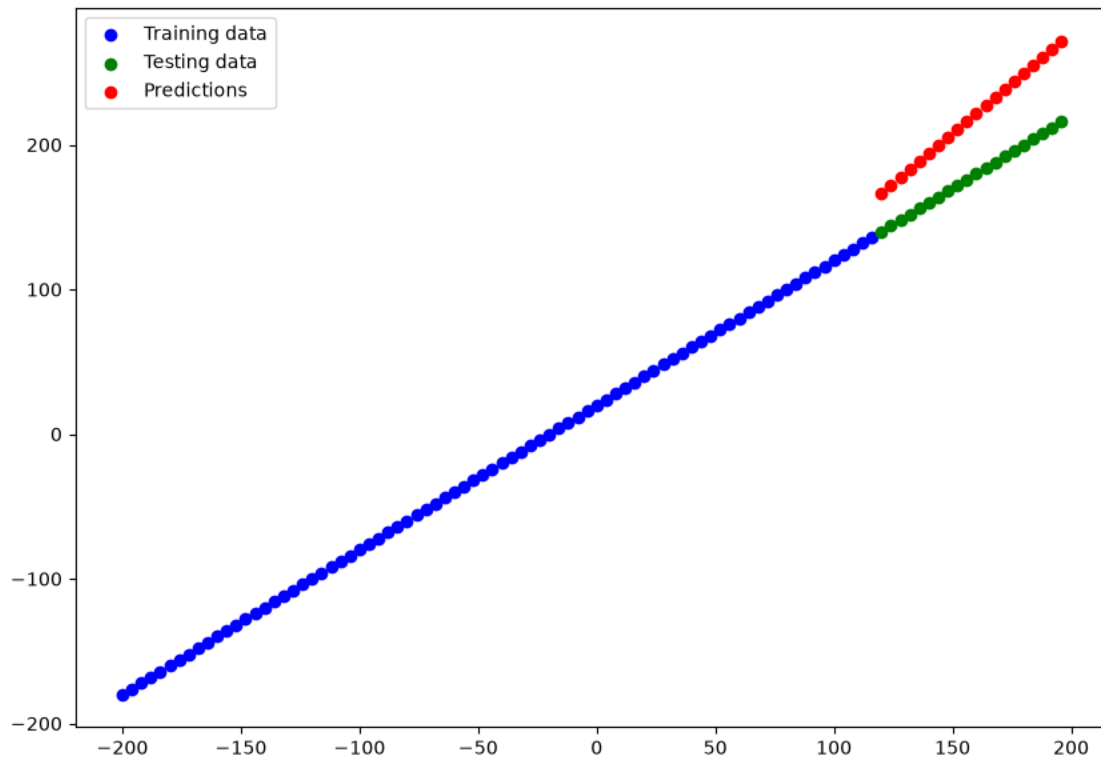
```
[19]: array([[166.50223],  
          [172.03183],  
          [177.56139],  
          [183.09097],  
          [188.62057],  
          [194.15016],  
          [199.67975],  
          [205.20935],  
          [210.73892],
```

```
[216.26851],  
[221.79811],  
[227.3277 ],  
[232.85728],  
[238.38689],  
[243.91646],  
[249.44604],  
[254.97565],  
[260.50525],  
[266.03482],  
[271.56442]], dtype=float32)
```

0.10 Plotting functions

```
[20]: def plot_predictions(train_data=X_train,  
                           train_labels=y_train,  
                           test_data=X_test,  
                           test_labels=y_test,  
                           predictions=y_preds):  
  
    """  
    Plots training data, test data and compares predictions.  
    """  
  
    plt.figure(figsize=(10, 7))  
    # Plot training data in blue  
    plt.scatter(train_data, train_labels, c="b", label="Training data")  
    # Plot test data in green  
    plt.scatter(test_data, test_labels, c="g", label="Testing data")  
    # Plot the predictions in red (predictions were made on the test data)  
    plt.scatter(test_data, predictions, c="r", label="Predictions")  
    # Show the legend  
    plt.legend();
```

```
[21]: plot_predictions(train_data=X_train,  
                       train_labels=y_train,  
                       test_data=X_test,  
                       test_labels=y_test,  
                       predictions=y_preds)
```



0.11 Getting model metrics

```
[22]: model.evaluate(X_test, y_test)
```

```
1/1          0s 436ms/step - loss:
41.0333 - mae: 41.0333
```

```
[22]: [41.03331756591797, 41.03331756591797]
```

```
[23]: # Calculate the mean absolute error
tf.metrics.mae(y_true=y_test, y_pred=y_preds)

# tf.metrics.mean_absolute_error into mae since its integrated
# in newer version of Keras
```

```
[23]: <tf.Tensor: shape=(20,), dtype=float32, numpy=
array([26.502228, 28.03183 , 29.561386, 31.090973, 32.620575, 34.15016 ,
       35.67975 , 37.20935 , 38.738922, 40.26851 , 41.79811 , 43.327698,
       44.857285, 46.386887, 47.91646 , 49.446045, 50.975647, 52.50525 ,
       54.03482 , 55.564423], dtype=float32)>
```

```
[24]: y_test, y_preds
```

```
[24]: (array([140, 144, 148, 152, 156, 160, 164, 168, 172, 176, 180, 184, 188,
          192, 196, 200, 204, 208, 212, 216]),
      array([[166.50223],
            [172.03183],
            [177.56139],
            [183.09097],
            [188.62057],
            [194.15016],
            [199.67975],
            [205.20935],
            [210.73892],
            [216.26851],
            [221.79811],
            [227.3277 ],
            [232.85728],
            [238.38689],
            [243.91646],
            [249.44604],
            [254.97565],
            [260.50525],
            [266.03482],
            [271.56442]]), dtype=float32))
```

```
[25]: # Check the tensor shapes
      y_test.shape, y_preds.shape
```

```
[25]: ((20,), (20, 1))
```

```
[26]: y_preds.squeeze().shape
```

```
[26]: (20,)
```

```
[27]: # What do they look like?
      y_test, y_preds.squeeze()
```

```
[27]: (array([140, 144, 148, 152, 156, 160, 164, 168, 172, 176, 180, 184, 188,
          192, 196, 200, 204, 208, 212, 216]),
      array([166.50223, 172.03183, 177.56139, 183.09097, 188.62057, 194.15016,
            199.67975, 205.20935, 210.73892, 216.26851, 221.79811, 227.3277 ,
            232.85728, 238.38689, 243.91646, 249.44604, 254.97565, 260.50525,
            266.03482, 271.56442], dtype=float32))
```

0.12 Testing eval metrics

```
[28]: # Calcuate the MAE
      tf.metrics.mae(y_true=y_test,
                    y_pred=y_preds.squeeze()) # use squeeze() to make same shape
```

```
[28]: <tf.Tensor: shape=(), dtype=float32, numpy=41.03331756591797>
```

```
[29]: # Calculate the MSE using pure TensorFlow math
mse = tf.reduce_mean(tf.square(y_test - y_preds.squeeze()))
mse
```

```
[29]: <tf.Tensor: shape=(), dtype=float64, numpy=1761.526097691746>
```

```
[30]: def mae(y_test, y_pred):
    """
    Calculuates mean absolute error between y_test and y_preds.
    """
    return tf.metrics.mean_absolute_error(y_test,
                                          y_pred)

def mse(y_test, y_pred):
    """
    Calculates mean squared error between y_test and y_preds.
    """
    return tf.metrics.mean_squared_error(y_test,
                                         y_pred)
```

0.13 Building New Models

0.14 Model No.1

```
[31]: # Set random seed
tf.random.set_seed(42)

# Replicate original model
model_1 = tf.keras.Sequential([
    tf.keras.layers.Dense(1)
])

# Compile the model
model_1.compile(loss=tf.keras.losses.mae,
                optimizer=tf.keras.optimizers.SGD(),
                metrics=['mae'])

# Fit the model
model_1.fit(tf.expand_dims(X_train, axis=-1), y_train, epochs=100)
```

Epoch 1/100

3/3 2s 36ms/step - loss:

46.9382 - mae: 46.9382

Epoch 2/100

3/3 0s 33ms/step - loss:

34.7322 - mae: 34.7322

Epoch 3/100

3/3 0s 37ms/step - loss:
34.8435 - mae: 34.8435
Epoch 4/100
3/3 0s 29ms/step - loss:
34.5982 - mae: 34.5982
Epoch 5/100
3/3 0s 27ms/step - loss:
34.6690 - mae: 34.6690
Epoch 6/100
3/3 0s 45ms/step - loss:
34.7476 - mae: 34.7476
Epoch 7/100
3/3 0s 27ms/step - loss:
34.5350 - mae: 34.5350
Epoch 8/100
3/3 0s 30ms/step - loss:
34.6059 - mae: 34.6059
Epoch 9/100
3/3 0s 27ms/step - loss:
34.6768 - mae: 34.6768
Epoch 10/100
3/3 0s 28ms/step - loss:
34.8159 - mae: 34.8159
Epoch 11/100
3/3 0s 34ms/step - loss:
34.5428 - mae: 34.5428
Epoch 12/100
3/3 0s 29ms/step - loss:
34.6137 - mae: 34.6137
Epoch 13/100
3/3 0s 30ms/step - loss:
34.7200 - mae: 34.7200
Epoch 14/100
3/3 0s 28ms/step - loss:
34.4797 - mae: 34.4797
Epoch 15/100
3/3 0s 50ms/step - loss:
34.5505 - mae: 34.5505
Epoch 16/100
3/3 0s 34ms/step - loss:
34.6242 - mae: 34.6242
Epoch 17/100
3/3 0s 26ms/step - loss:
34.4165 - mae: 34.4165
Epoch 18/100
3/3 0s 28ms/step - loss:
34.4874 - mae: 34.4874
Epoch 19/100

3/3 0s 28ms/step - loss:
34.5583 - mae: 34.5583
Epoch 20/100
3/3 0s 28ms/step - loss:
34.6924 - mae: 34.6924
Epoch 21/100
3/3 0s 30ms/step - loss:
34.4243 - mae: 34.4243
Epoch 22/100
3/3 0s 27ms/step - loss:
34.4952 - mae: 34.4952
Epoch 23/100
3/3 0s 27ms/step - loss:
34.5966 - mae: 34.5966
Epoch 24/100
3/3 0s 67ms/step - loss:
34.3611 - mae: 34.3611
Epoch 25/100
3/3 1s 184ms/step - loss:
34.4320 - mae: 34.4320
Epoch 26/100
3/3 0s 63ms/step - loss:
34.5029 - mae: 34.5029
Epoch 27/100
3/3 0s 34ms/step - loss:
34.6648 - mae: 34.6648
Epoch 28/100
3/3 0s 26ms/step - loss:
34.3689 - mae: 34.3689
Epoch 29/100
3/3 0s 38ms/step - loss:
34.4398 - mae: 34.4398
Epoch 30/100
3/3 0s 33ms/step - loss:
34.5690 - mae: 34.5690
Epoch 31/100
3/3 0s 32ms/step - loss:
34.3058 - mae: 34.3058
Epoch 32/100
3/3 0s 30ms/step - loss:
34.3766 - mae: 34.3766
Epoch 33/100
3/3 0s 32ms/step - loss:
34.4731 - mae: 34.4731
Epoch 34/100
3/3 0s 128ms/step - loss:
34.2426 - mae: 34.2426
Epoch 35/100

3/3 0s 100ms/step - loss:
34.3135 - mae: 34.3135
Epoch 36/100
3/3 0s 43ms/step - loss:
34.3844 - mae: 34.3844
Epoch 37/100
3/3 0s 28ms/step - loss:
34.5414 - mae: 34.5414
Epoch 38/100
3/3 0s 36ms/step - loss:
34.2504 - mae: 34.2504
Epoch 39/100
3/3 1s 64ms/step - loss:
34.3213 - mae: 34.3213
Epoch 40/100
3/3 0s 50ms/step - loss:
34.4455 - mae: 34.4455
Epoch 41/100
3/3 0s 47ms/step - loss:
34.1873 - mae: 34.1873
Epoch 42/100
3/3 0s 52ms/step - loss:
34.2581 - mae: 34.2581
Epoch 43/100
3/3 0s 50ms/step - loss:
34.3497 - mae: 34.3497
Epoch 44/100
3/3 0s 38ms/step - loss:
34.1241 - mae: 34.1241
Epoch 45/100
3/3 0s 58ms/step - loss:
34.1950 - mae: 34.1950
Epoch 46/100
3/3 1s 105ms/step - loss:
34.2659 - mae: 34.2659
Epoch 47/100
3/3 0s 42ms/step - loss:
34.4180 - mae: 34.4180
Epoch 48/100
3/3 0s 34ms/step - loss:
34.1319 - mae: 34.1319
Epoch 49/100
3/3 0s 45ms/step - loss:
34.2028 - mae: 34.2028
Epoch 50/100
3/3 0s 70ms/step - loss:
34.3221 - mae: 34.3221
Epoch 51/100

3/3 0s 49ms/step - loss:
 34.0688 - mae: 34.0688
 Epoch 52/100
 3/3 0s 86ms/step - loss:
 34.1396 - mae: 34.1396
 Epoch 53/100
 3/3 0s 39ms/step - loss:
 34.2262 - mae: 34.2262
 Epoch 54/100
 3/3 0s 53ms/step - loss:
 34.0056 - mae: 34.0056
 Epoch 55/100
 3/3 1s 36ms/step - loss:
 34.0765 - mae: 34.0765
 Epoch 56/100
 3/3 0s 100ms/step - loss:
 34.1474 - mae: 34.1474
 Epoch 57/100
 3/3 0s 45ms/step - loss:
 34.2945 - mae: 34.2945
 Epoch 58/100
 3/3 0s 43ms/step - loss:
 34.0134 - mae: 34.0134
 Epoch 59/100
 3/3 0s 36ms/step - loss:
 34.0843 - mae: 34.0843
 Epoch 60/100
 3/3 0s 59ms/step - loss:
 34.1987 - mae: 34.1987
 Epoch 61/100
 3/3 0s 39ms/step - loss:
 33.9503 - mae: 33.9503
 Epoch 62/100
 3/3 0s 40ms/step - loss:
 34.0211 - mae: 34.0211
 Epoch 63/100
 3/3 0s 111ms/step - loss:
 34.1028 - mae: 34.1028
 Epoch 64/100
 3/3 0s 51ms/step - loss:
 33.8871 - mae: 33.8871
 Epoch 65/100
 3/3 0s 35ms/step - loss:
 33.9580 - mae: 33.9580
 Epoch 66/100
 3/3 0s 49ms/step - loss:
 34.0289 - mae: 34.0289
 Epoch 67/100

3/3 0s 52ms/step - loss:
34.1711 - mae: 34.1711
Epoch 68/100
3/3 0s 31ms/step - loss:
33.8949 - mae: 33.8949
Epoch 69/100
3/3 0s 63ms/step - loss:
33.9658 - mae: 33.9658
Epoch 70/100
3/3 0s 48ms/step - loss:
34.0752 - mae: 34.0752
Epoch 71/100
3/3 0s 54ms/step - loss:
33.8318 - mae: 33.8318
Epoch 72/100
3/3 0s 43ms/step - loss:
33.9026 - mae: 33.9026
Epoch 73/100
3/3 0s 45ms/step - loss:
33.9794 - mae: 33.9794
Epoch 74/100
3/3 0s 91ms/step - loss:
33.7686 - mae: 33.7686
Epoch 75/100
3/3 0s 49ms/step - loss:
33.8395 - mae: 33.8395
Epoch 76/100
3/3 0s 49ms/step - loss:
33.9104 - mae: 33.9104
Epoch 77/100
3/3 0s 36ms/step - loss:
34.0730 - mae: 34.0730
Epoch 78/100
3/3 0s 54ms/step - loss:
33.7383 - mae: 33.7383
Epoch 79/100
3/3 0s 32ms/step - loss:
33.8091 - mae: 33.8091
Epoch 80/100
3/3 0s 45ms/step - loss:
33.8800 - mae: 33.8800
Epoch 81/100
3/3 0s 38ms/step - loss:
34.0666 - mae: 34.0666
Epoch 82/100
3/3 0s 27ms/step - loss:
33.7079 - mae: 33.7079
Epoch 83/100

3/3 0s 29ms/step - loss:
33.7788 - mae: 33.7788
Epoch 84/100
3/3 0s 74ms/step - loss:
33.8563 - mae: 33.8563
Epoch 85/100
3/3 0s 28ms/step - loss:
33.6448 - mae: 33.6448
Epoch 86/100
3/3 0s 28ms/step - loss:
33.7156 - mae: 33.7156
Epoch 87/100
3/3 0s 29ms/step - loss:
33.7865 - mae: 33.7865
Epoch 88/100
3/3 0s 29ms/step - loss:
33.9571 - mae: 33.9571
Epoch 89/100
3/3 0s 47ms/step - loss:
33.6144 - mae: 33.6144
Epoch 90/100
3/3 0s 32ms/step - loss:
33.6853 - mae: 33.6853
Epoch 91/100
3/3 0s 28ms/step - loss:
33.7561 - mae: 33.7561
Epoch 92/100
3/3 0s 30ms/step - loss:
33.9507 - mae: 33.9507
Epoch 93/100
3/3 0s 27ms/step - loss:
33.5840 - mae: 33.5840
Epoch 94/100
3/3 0s 28ms/step - loss:
33.6549 - mae: 33.6549
Epoch 95/100
3/3 0s 82ms/step - loss:
33.7333 - mae: 33.7333
Epoch 96/100
3/3 0s 52ms/step - loss:
33.5209 - mae: 33.5209
Epoch 97/100
3/3 0s 69ms/step - loss:
33.5918 - mae: 33.5918
Epoch 98/100
3/3 0s 43ms/step - loss:
33.6626 - mae: 33.6626
Epoch 99/100

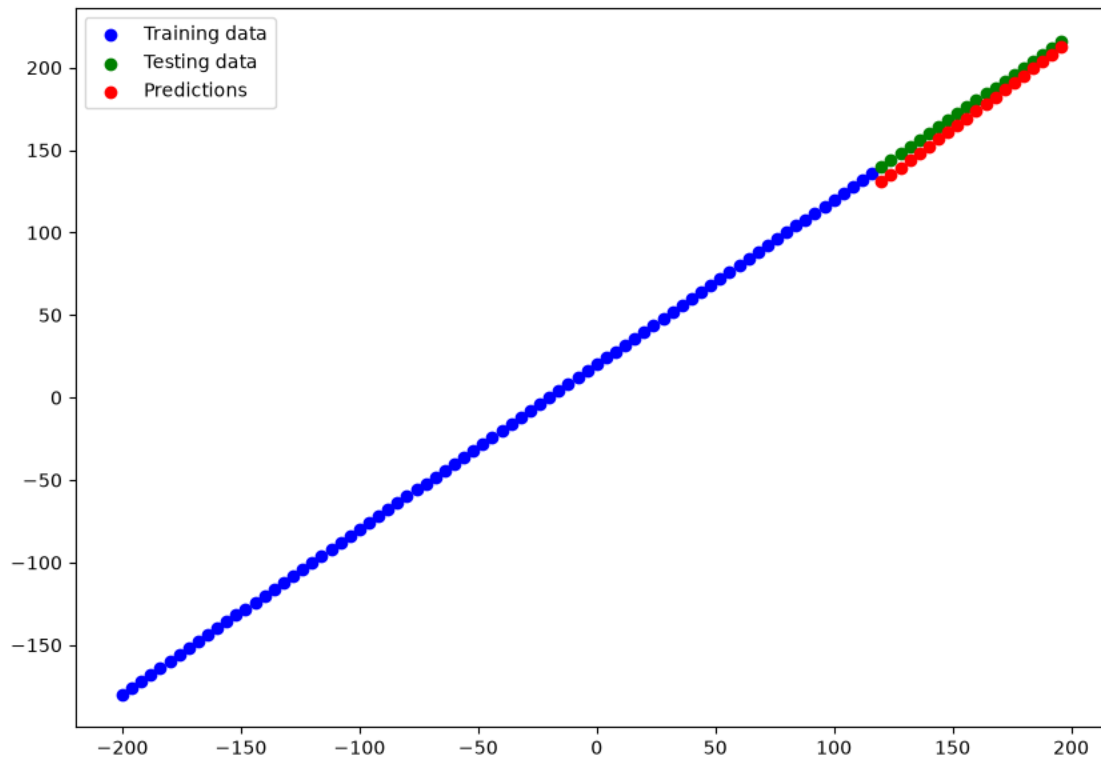
```
3/3          0s 54ms/step - loss:
33.8412 - mae: 33.8412
Epoch 100/100
3/3          0s 35ms/step - loss:
33.4905 - mae: 33.4905
```

```
[31]: <keras.src.callbacks.history.History at 0x2047e5e4da0>
```

0.15 Visuals of model no.1

```
[32]: # Make and plot predictions for model_1
y_preds_1 = model_1.predict(X_test)
plot_predictions(predictions=y_preds_1)
```

```
1/1          0s 211ms/step
```



0.16 Fix for Calculation of mse & mae

```
[33]: def mae(y_test, y_pred):
      """
      Calculates mean absolute error between y_test and y_preds.
      """
      return tf.metrics.mae(y_test, y_pred)
```

```
def mse(y_test, y_pred):
    """
    Calculates mean squared error between y_test and y_preds.
    """
    # Using the built-in shortened alias
    return tf.metrics.mse(y_test, y_pred)

    # Alternatively, if the above still gives version issues, use pure TF math:
    # return tf.reduce_mean(tf.square(y_test - y_pred))
```

```
[34]: mae_1 = mae(y_test, y_preds_1.squeeze()).numpy()
      mse_1 = mse(y_test, y_preds_1.squeeze()).numpy()
      mae_1, mse_1
```

```
[34]: (np.float32(6.437018), np.float32(44.30239))
```

0.17 Making model no. 2

```
[ ]: # Set random seed
      tf.random.set_seed(42)

      # Replicate model_1, add an extra layer and set first layer to have 100 hidden_
      ↪units
      model_2 = tf.keras.Sequential([
          tf.keras.layers.Dense(100), #set hidden units to 100
          tf.keras.layers.Dense(1) # add a second layer
      ])

      # Compile the model
      model_2.compile(loss=tf.keras.losses.mae,
                      optimizer=tf.keras.optimizers.Adam(learning_rate=0.001), #
                      ↪change optimizer to Adam with a learning rate of 0.001
                      metrics=['mae'])

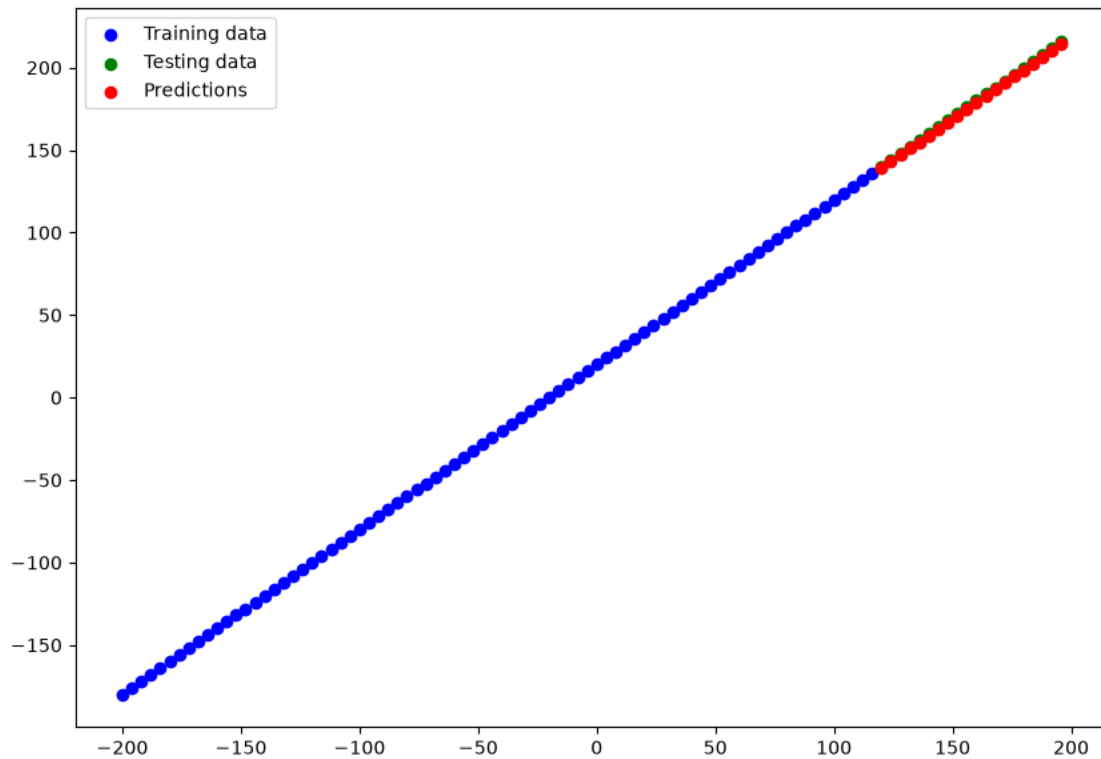
      # Fit the model
      model_2.fit(tf.expand_dims(X_train, axis=-1), y_train, epochs=500, verbose=0) #
      ↪set verbose to 0 for less output
```

```
[ ]: <keras.src.callbacks.history.History at 0x2047e4c5e80>
```

0.18 Visuals of model no.2

```
[36]: # Make and plot predictions for model_2
      y_preds_2 = model_2.predict(X_test)
      plot_predictions(predictions=y_preds_2)
```

1/1 0s 155ms/step



```
[37]: # Calculate model_2 metrics
mae_2 = mae(y_test, y_preds_2.squeeze()).numpy()
mse_2 = mse(y_test, y_preds_2.squeeze()).numpy()
mae_2, mse_2
```

```
[37]: (np.float32(1.2965645), np.float32(1.7168621))
```

0.19 Making model no. 3

```
[38]: # Set random seed
tf.random.set_seed(42)

# Replicate model_2 and add an extra layer with 100 hidden units
model_3 = tf.keras.Sequential([
    tf.keras.layers.Dense(100),
    tf.keras.layers.Dense(100), #adds second layer with 100 hidden units
    tf.keras.layers.Dense(1)
])

# Compile the model
model_3.compile(loss=tf.keras.losses.mae,
                optimizer=tf.keras.optimizers.Adam(learning_rate=0.001),
```

```

        metrics=['mae'])

# Fit the model
model_3.fit(tf.expand_dims(X_train, axis=-1), y_train, epochs=500, verbose=0)

```

[38]: <keras.src.callbacks.history.History at 0x204035ec4d0>

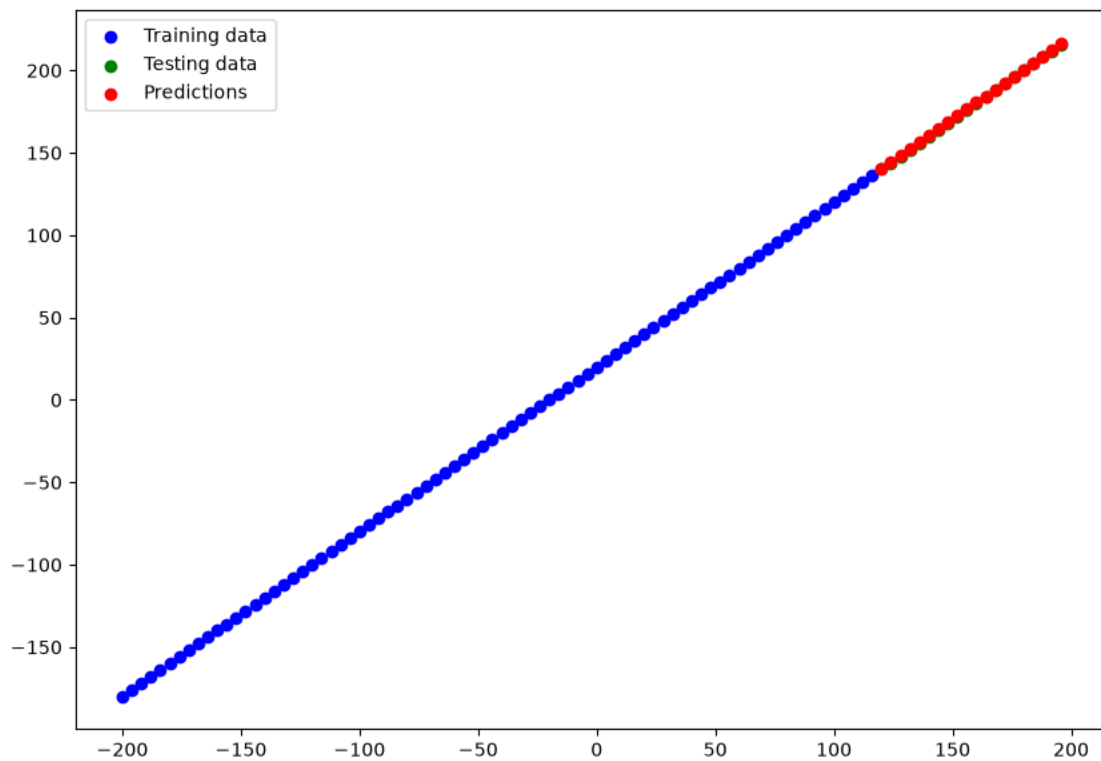
0.20 Visuals of model no.3

```

[39]: # Make and plot predictions for model_3
y_preds_3 = model_3.predict(X_test)
plot_predictions(predictions=y_preds_3)

```

1/1 0s 180ms/step



```

[40]: # Calculate model_3 metrics
mae_3 = mae(y_test, y_preds_3.squeeze()).numpy()
mse_3 = mse(y_test, y_preds_3.squeeze()).numpy()
mae_3, mse_3

```

[40]: (np.float32(0.45003587), np.float32(0.20681004))

0.21 Making model no. 4

```
[41]: # Set random seed
tf.random.set_seed(42)

# Replicate model_3
model_4 = tf.keras.Sequential([
    tf.keras.layers.Dense(100),
    tf.keras.layers.Dense(100),
    tf.keras.layers.Dense(1)
])

# Compile the model
model_4.compile(loss=tf.keras.losses.mae,
                optimizer=tf.keras.optimizers.Adam(learning_rate=0.0001), #set_
↪ learning rate to 0.0001
                metrics=['mae'])

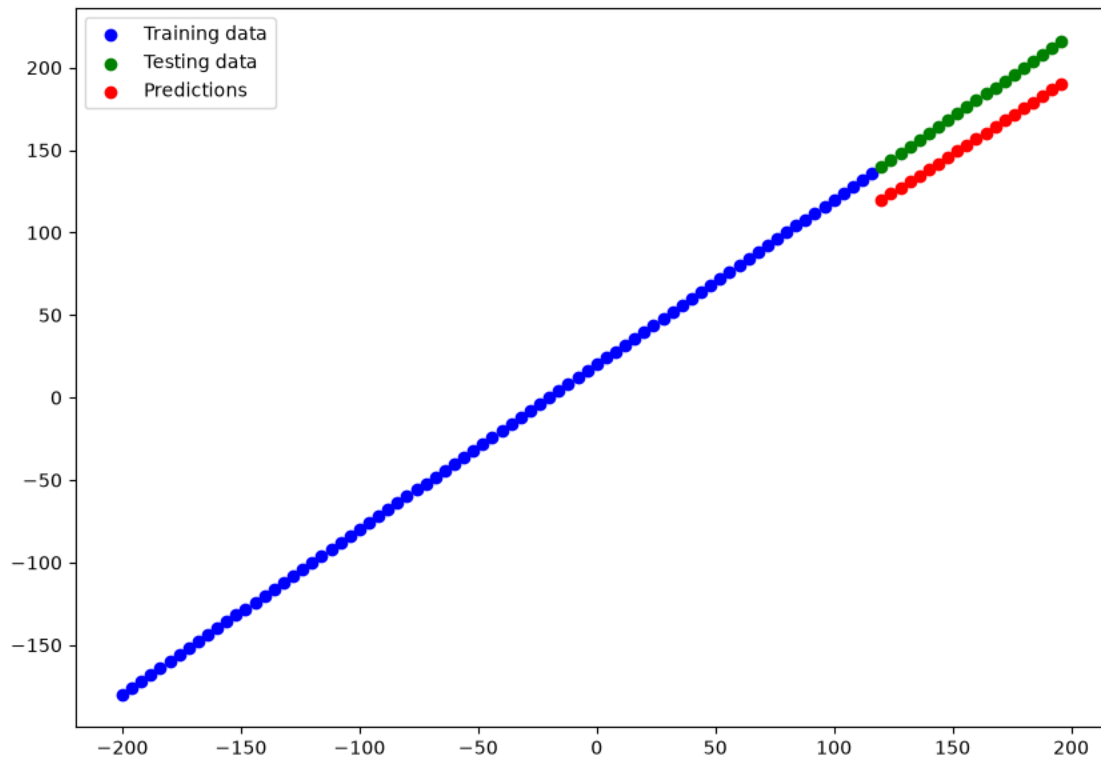
# Fit the model
model_4.fit(tf.expand_dims(X_train, axis=-1), y_train, epochs=500, verbose=0)
```

```
[41]: <keras.src.callbacks.history.History at 0x2047e5f0380>
```

0.22 Visuals of model no.4

```
[42]: # Make and plot predictions for model_4
y_preds_4 = model_4.predict(X_test)
plot_predictions(predictions=y_preds_4)
```

```
WARNING:tensorflow:5 out of the last 5 calls to <function
TensorFlowTrainer.make_predict_function.<locals>.one_step_on_data_distributed at
0x000002047E2D80E0> triggered tf.function retracing. Tracing is expensive and
the excessive number of tracings could be due to (1) creating @tf.function
repeatedly in a loop, (2) passing tensors with different shapes, (3) passing
Python objects instead of tensors. For (1), please define your @tf.function
outside of the loop. For (2), @tf.function has reduce_retracing=True option that
can avoid unnecessary retracing. For (3), please refer to
https://www.tensorflow.org/guide/function#controlling\_retracing and
https://www.tensorflow.org/api\_docs/python/tf/function for more details.
1/1          0s 465ms/step
```



```
[43]: # Calculate model_4 metrics
mae_4 = mae(y_test, y_preds_4.squeeze()).numpy()
mse_4 = mse(y_test, y_preds_4.squeeze()).numpy()
mae_4, mse_4
```

```
[43]: (np.float32(23.06998), np.float32(534.9774))
```

0.23 Making model no. 5

```
[44]: # Set random seed
tf.random.set_seed(42)

# Replicate model_3
model_5 = tf.keras.Sequential([
    tf.keras.layers.Dense(100),
    tf.keras.layers.Dense(100),
    tf.keras.layers.Dense(1)
])

# Compile the model
model_5.compile(loss=tf.keras.losses.mae,
```

```

optimizer=tf.keras.optimizers.Adam(learning_rate=0.001), #set_
↪learning rate to 0.001
metrics=['mae'])

# Fit the model
model_5.fit(tf.expand_dims(X_train, axis=-1), y_train, epochs=500, verbose=0)

```

[44]: <keras.src.callbacks.history.History at 0x20403040e00>

0.24 Visuals of model no.5

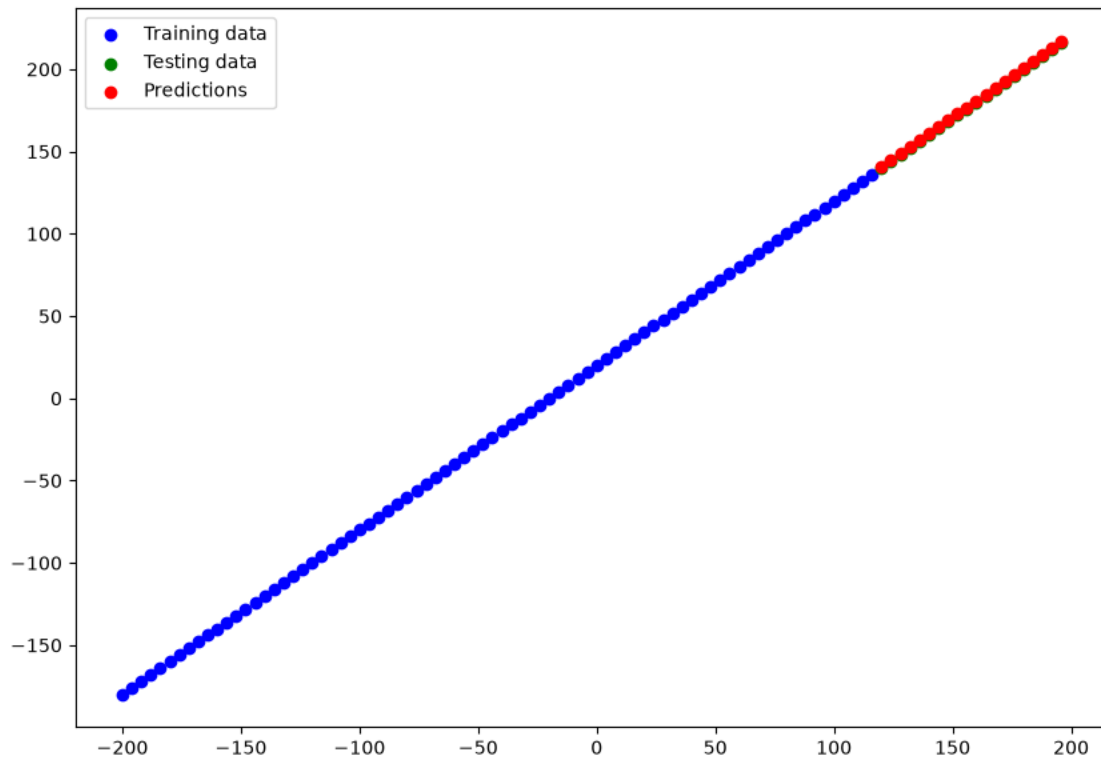
```

[45]: # Make and plot predictions for model_5
y_preds_5 = model_5.predict(X_test)
plot_predictions(predictions=y_preds_5)

```

WARNING:tensorflow:6 out of the last 6 calls to <function TensorFlowTrainer.make_predict_function.<locals>.one_step_on_data_distributed at 0x0000020404A2E480> triggered tf.function retracing. Tracing is expensive and the excessive number of tracings could be due to (1) creating @tf.function repeatedly in a loop, (2) passing tensors with different shapes, (3) passing Python objects instead of tensors. For (1), please define your @tf.function outside of the loop. For (2), @tf.function has reduce_retracing=True option that can avoid unnecessary retracing. For (3), please refer to https://www.tensorflow.org/guide/function#controlling_retracing and https://www.tensorflow.org/api_docs/python/tf/function for more details.

1/1 0s 252ms/step



```
[46]: # Calculate model_5 metrics
mae_5 = mae(y_test, y_preds_5.squeeze()).numpy()
mse_5 = mse(y_test, y_preds_5.squeeze()).numpy()
mae_5, mse_5
```

```
[46]: (np.float32(0.8456047), np.float32(0.7305636))
```

0.25 Making model no. 6

```
[47]: # Set random seed
tf.random.set_seed(42)

# Replicate model_3
model_6 = tf.keras.Sequential([
    tf.keras.layers.Dense(100),
    tf.keras.layers.Dense(100),
    tf.keras.layers.Dense(100),
    tf.keras.layers.Dense(1)
])

# Compile the model
model_6.compile(loss=tf.keras.losses.mae,
```

```

optimizer=tf.keras.optimizers.Adam(learning_rate=0.0001), #set_
↪learning rate to 0.0001
metrics=['mae'])

# Fit the model
model_6.fit(tf.expand_dims(X_train, axis=-1), y_train, epochs=500, verbose=0)

```

[47]: <keras.src.callbacks.history.History at 0x204030d8d40>

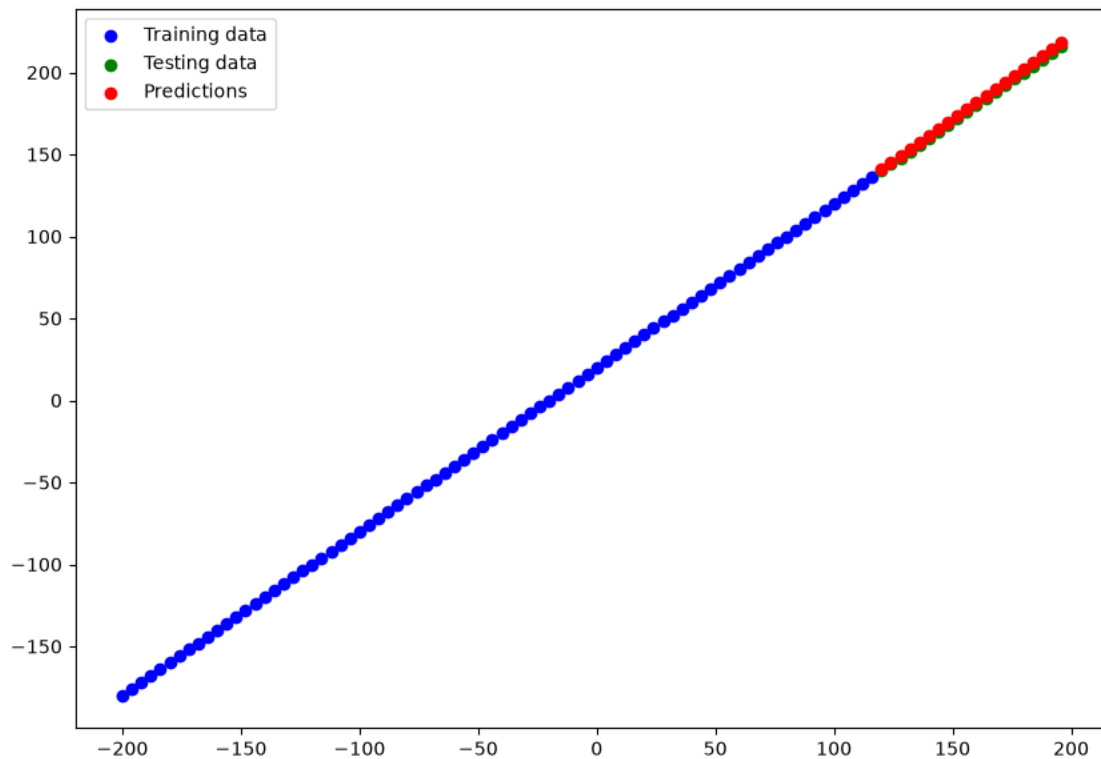
0.26 Visuals of model no.6

```

[48]: # Make and plot predictions for model_6
y_preds_6 = model_6.predict(X_test)
plot_predictions(predictions=y_preds_6)

```

1/1 0s 172ms/step



```

[49]: # Calculate model_6 metrics
mae_6 = mae(y_test, y_preds_6.squeeze()).numpy()
mse_6 = mse(y_test, y_preds_6.squeeze()).numpy()
mae_6, mse_6

```

[49]: (np.float32(1.9661682), np.float32(3.9494014))

0.27 Making model no. 7

```
[50]: # Set random seed
tf.random.set_seed(42)

# Replicate model_6
model_7 = tf.keras.Sequential([
    tf.keras.layers.Dense(100),
    tf.keras.layers.Dense(100),
    tf.keras.layers.Dense(100),
    tf.keras.layers.Dense(1)
])

# Compile the model
model_7.compile(loss=tf.keras.losses.mae,
                 optimizer=tf.keras.optimizers.Adam(learning_rate=0.001), #set_
                 ↪learning rate to 0.001
                 metrics=['mae'])

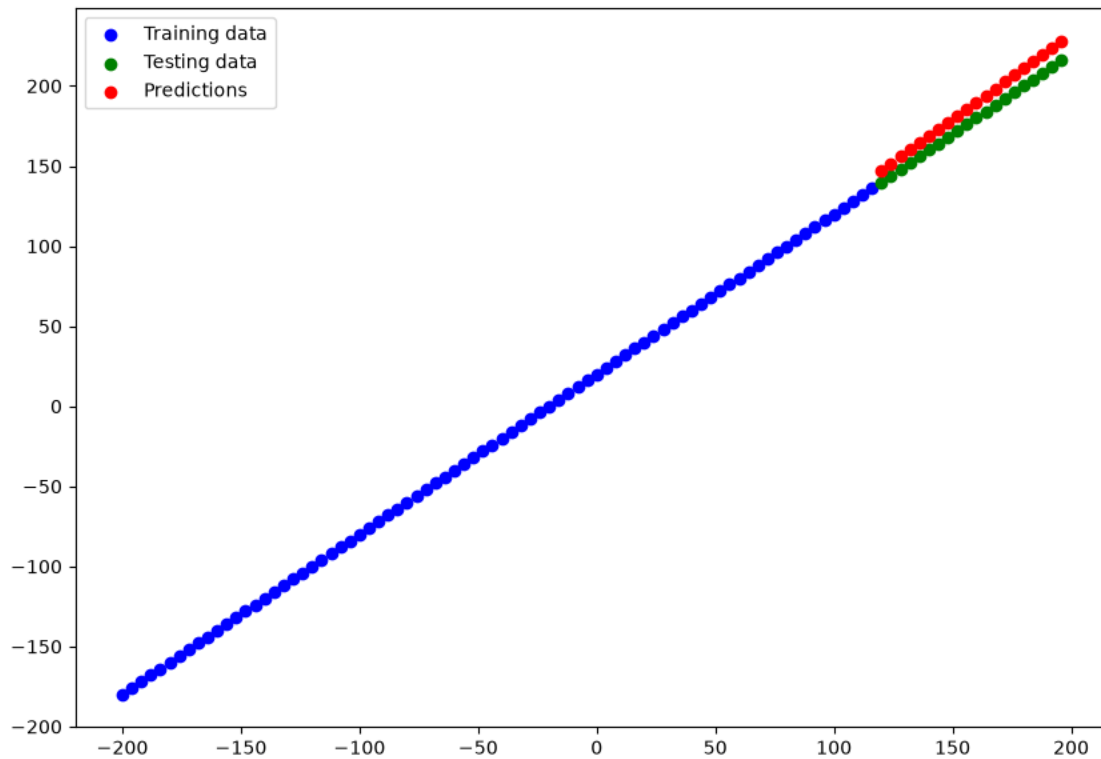
# Fit the model
model_7.fit(tf.expand_dims(X_train, axis=-1), y_train, epochs=500, verbose=0)
```

```
[50]: <keras.src.callbacks.history.History at 0x20404c803e0>
```

0.28 Visuals of model no.7

```
[51]: # Make and plot predictions for model_7
y_preds_7 = model_7.predict(X_test)
plot_predictions(predictions=y_preds_7)
```

```
1/1          0s 182ms/step
```



```
[52]: # Calculate model_7 metrics
mae_7 = mae(y_test, y_preds_7.squeeze()).numpy()
mse_7 = mse(y_test, y_preds_7.squeeze()).numpy()
mae_7, mse_7
```

```
[52]: (np.float32(9.676714), np.float32(95.52132))
```

0.29 Making model no. 8

```
[53]: # Set random seed
tf.random.set_seed(42)

# Replicate model_5
model_8 = tf.keras.Sequential([
    tf.keras.layers.Dense(100),
    tf.keras.layers.Dense(100),
    tf.keras.layers.Dense(1)
])

# Compile the model
model_8.compile(loss=tf.keras.losses.mae,
                 optimizer=tf.keras.optimizers.Adam(learning_rate=0.001),
```

```

        metrics=['mae'])

# Fit the model
model_8.fit(tf.expand_dims(X_train, axis=-1), y_train, epochs=500, verbose=0)

```

[53]: <keras.src.callbacks.history.History at 0x20403721280>

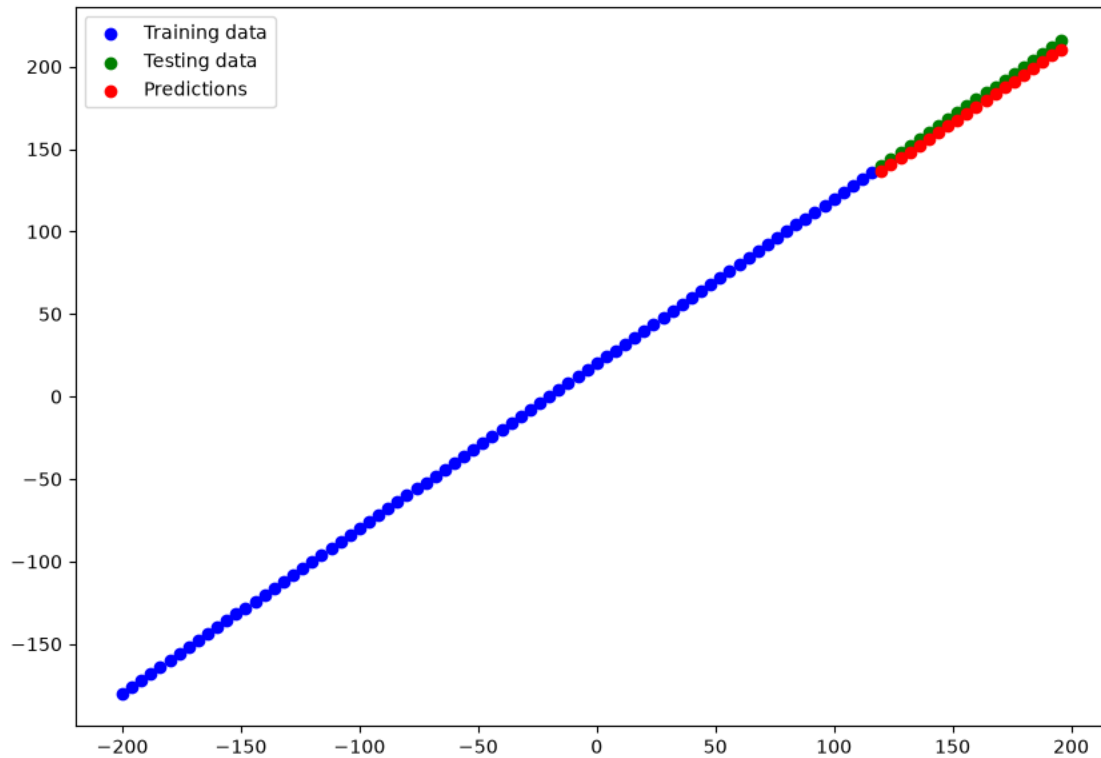
0.30 Visuals of model no.8

```

[54]: # Make and plot predictions for model_8
y_preds_8 = model_8.predict(X_test)
plot_predictions(predictions=y_preds_8)

```

1/1 0s 203ms/step



```

[55]: # Calculate model_8 metrics
mae_8 = mae(y_test, y_preds_8.squeeze()).numpy()
mse_8 = mse(y_test, y_preds_8.squeeze()).numpy()
mae_8, mse_8

```

[55]: (np.float32(4.3573256), np.float32(19.38425))

0.31 Rerun model no. 8

```
[56]: # Set random seed
tf.random.set_seed(42)

# Replicate model_5
model_8 = tf.keras.Sequential([
    tf.keras.layers.Dense(100),
    tf.keras.layers.Dense(100),
    tf.keras.layers.Dense(1)
])

# Compile the model
model_8.compile(loss=tf.keras.losses.mae,
                optimizer=tf.keras.optimizers.Adam(learning_rate=0.001),
                metrics=['mae'])

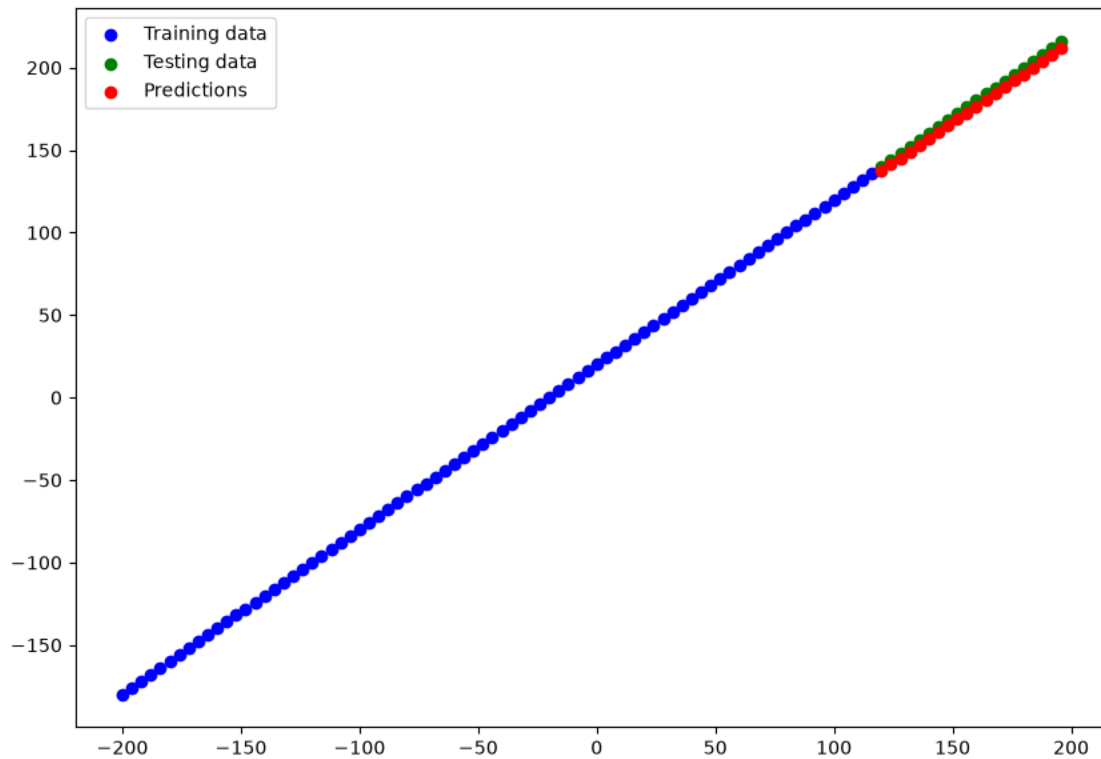
# Fit the model
model_8.fit(tf.expand_dims(X_train, axis=-1), y_train, epochs=500, verbose=0)
```

```
[56]: <keras.src.callbacks.history.History at 0x20409bee480>
```

0.32 Visuals of model no.8

```
[57]: # Make and plot predictions for model_8
y_preds_8 = model_8.predict(X_test)
plot_predictions(predictions=y_preds_8)
```

```
1/1          0s 234ms/step
```



```
[58]: # Calculate model_8 metrics
mae_8 = mae(y_test, y_preds_8.squeeze()).numpy()
mse_8 = mse(y_test, y_preds_8.squeeze()).numpy()
mae_8, mse_8
```

```
[58]: (np.float32(3.4524384), np.float32(12.165149))
```

0.33 Rerun model no. 8 with more epochs

```
[59]: # Set random seed
tf.random.set_seed(42)

# Replicate model_5
model_8 = tf.keras.Sequential([
    tf.keras.layers.Dense(100),
    tf.keras.layers.Dense(100),
    tf.keras.layers.Dense(1)
])

# Compile the model
model_8.compile(loss=tf.keras.losses.mae,
                 optimizer=tf.keras.optimizers.Adam(learning_rate=0.001),
```

```

        metrics=['mae'])

# Fit the model
model_8.fit(tf.expand_dims(X_train, axis=-1), y_train, epochs=750, verbose=0)

```

[59]: <keras.src.callbacks.history.History at 0x2040adaa8d0>

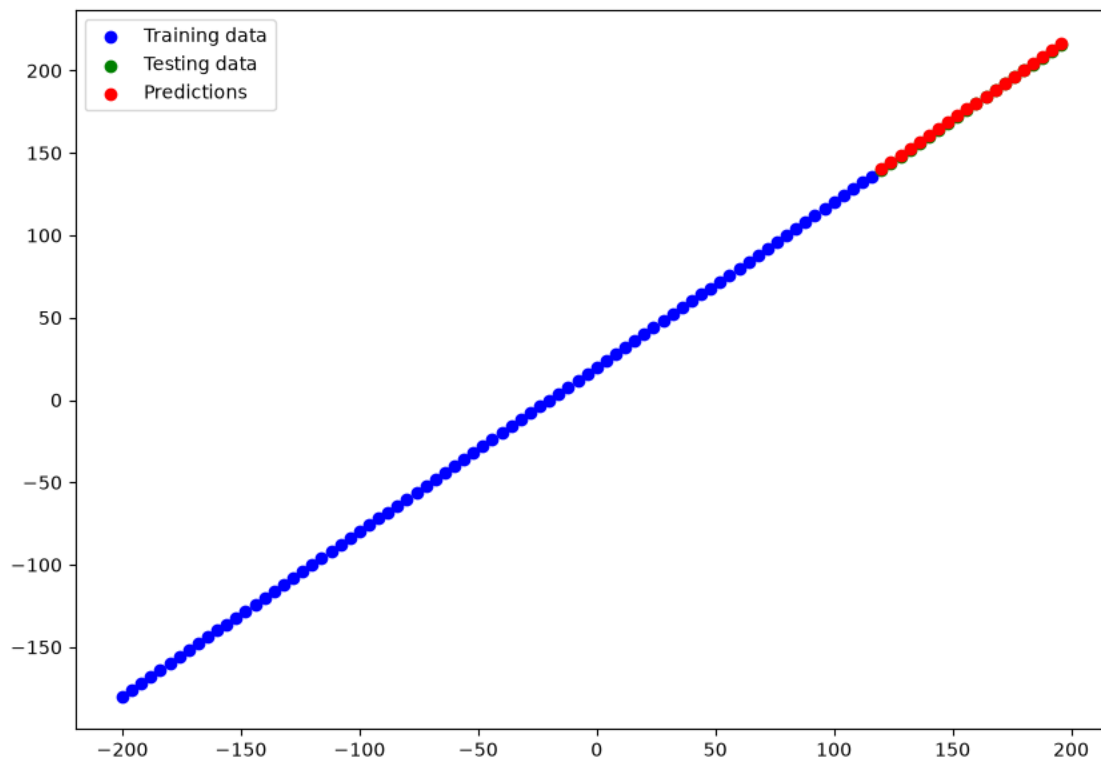
0.34 Visuals of model no.8

```

[60]: # Make and plot predictions for model_8
y_preds_8 = model_8.predict(X_test)
plot_predictions(predictions=y_preds_8)

```

1/1 0s 209ms/step



```

[61]: # Calculate model_8 metrics
mae_8 = mae(y_test, y_preds_8.squeeze()).numpy()
mse_8 = mse(y_test, y_preds_8.squeeze()).numpy()
mae_8, mse_8

```

[61]: (np.float32(0.5200302), np.float32(0.27660736))